

141. Linked List Cycle

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: June 19, 2026

Problem

Given the head of a linked list, determine whether the linked list contains a cycle.

A cycle exists if some node can be reached again by continuously following next pointers.

Approach

Use two pointers:

- Slow
- fast

slow moves 1 step at a time.

fast moves 2 steps at a time.

This is called: Floyd's Cycle Detection Algorithm (Tortoise and Hare)

Time Complexity & Space Complexity

- $O(n)$ Each pointer traverses at most a linear number of nodes.
- $O(1)$ No extra data structures used.

Key Insight

Since the list is **sorted**, duplicates are always **adjacent**. So we never need to look ahead more than one node — just compare each node with the previous one and skip if equal.

Algorithm

Handle empty list.

Initialize:

slow = head

fast = head.next

While pointers are different:

If fast reaches null:

return false;

Move:

slow = slow.next;

fast = fast.next.next;

If pointers meet:

return true;

141. Linked List Cycle

Solved

Easy

Topics

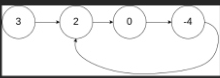
Companies

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:



Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:



Input: head = [1,2], pos = 0

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Accepted

29 / 29 testcases passed

Sahil Khan submitted at Jun 06, 2025 11:52

Runtime

0 ms | Beats 100.00%

Memory

44.54 MB | Beats 99.97%

Code : Java

```
1 /**
2  * Definition for singly-linked list.
3  * class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode(int x) {
7  *         val = x;
8  *         next = null;
9  *     }
10  * }
11  */
12 public class Solution {
13     public boolean hasCycle(ListNode head) {
14         if(head==null || head.next==null) return false;
15         ListNode slow = head;
16         ListNode fast = head.next;
17
18         while(slow != fast){
19             if(fast==null || fast.next==null){
20                 return false;
21             }
22             slow = slow.next;
23             fast = fast.next.next;
24         }
25
26         return true;
27     }
28 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

head =

[1]

pos =

-1